

Checklist:

The Well-Rounded Senior Engineer

Well-Rounded Senior Engineer Checklist

1. Think like a software engineer, not only a developer

Long-term engineering mindset

- ☐ I think beyond writing code and consider the system's long-term maintainability.
- ☐ I consider how my work will be extended, operated, debugged, monitored, and migrated later.
- ☐ I choose solutions that help the codebase remain sustainable over time.
- ☐ I look for ways to prevent recurring bugs, not just fix them once.
- ☐ I document or explain decisions that future engineers will need to understand.
- ☐ I consider whether a short-term shortcut creates avoidable long-term cost.

Senior-level expectations

- ☐ I can independently handle medium-sized or complex projects.
- ☐ I unblock myself in most situations.
- ☐ I take initiative for work within my scope.
- ☐ I follow team engineering practices and improve them when useful.
- ☐ I design project architecture and seek feedback before locking it in.
- ☐ I collaborate effectively with engineers and stakeholders.
- ☐ I mentor less experienced engineers while also seeking mentorship.

- ☐ I keep learning new domains, platforms, tools, and practices.

2. Getting things done

Communicating progress and complexity

- ☐ I communicate what I did, why it mattered, and why it was difficult.
- ☐ I make hidden complexity visible to my manager, peers, and stakeholders.
- ☐ I share meaningful progress updates, not just “done” or “not done.”
- ☐ I explain tradeoffs, risks, and decisions clearly.
- ☐ I communicate roadblocks early instead of disappearing into them.
- ☐ I offer options when blocked, such as reducing scope, using a temporary workaround, or asking another team for help.
- ☐ I under-promise, over-deliver, and over-communicate.

Managing my own work

- ☐ I know my top priority at all times.
- ☐ I protect time for uninterrupted deep work.
- ☐ I say “not right now” when I am in the zone.
- ☐ I timebox help requests so I do not lose entire days to interruptions.
- ☐ I turn synchronous requests into asynchronous ones when appropriate.
- ☐ I redirect requests to someone who would benefit from handling them.
- ☐ I regularly purge stale or low-value tasks from my backlog.
- ☐ I know what is normal to feel overwhelmed by, and I escalate before overload becomes failure.

Prioritizing requests

- ☐ I do urgent and important work immediately.
- ☐ I backlog important but non-urgent work.
- ☐ I pass on, delegate, or say no to urgent but unimportant work.
- ☐ I say no to work that is neither urgent nor important.
- ☐ I keep a lightweight system for capturing important non-urgent work.
- ☐ I review whether old “important” tasks are still important.

Doing work properly

- ☐ I write or confirm a specification before building non-trivial features.
- ☐ I clarify edge cases and out-of-scope behavior before implementation.
- ☐ I create a test plan before or during development.
- ☐ I include testing, monitoring, and alerting in my estimates.
- ☐ I do not throw work “over the fence” to QA.
- ☐ I involve QA or testing expertise early when available.
- ☐ I ship in short iterations when possible.
- ☐ I break large work into smaller, reviewable, shippable pieces.
- ☐ I recognize when longer stretches of work are justified, such as research, infrastructure, major refactors, or rewrites.
- ☐ I seek feedback before a long-running branch or project becomes too hard to change.

Helping the team get things done

- ☐ I break down and clearly estimate projects.
- ☐ I write notes, diagrams, or documentation so others do not need repeated explanations.
- ☐ I notice when teammates are blocked.
- ☐ I pair with teammates who are blocked, rather than simply telling them the answer.
- ☐ I help resolve external blockers by contacting other teams or stakeholders.
- ☐ I build relationships that make future cross-team work easier.
- ☐ I look for creative, unconventional solutions when obvious approaches are stuck.

Product and business awareness

- ☐ I understand how my company succeeds.
- ☐ I understand the product, customers, and business goals behind my work.
- ☐ I build a relationship with the product manager or business stakeholder.
- ☐ I bring product-informed suggestions to the table.
- ☐ I make product/engineering tradeoffs explicit.
- ☐ I seek frequent product feedback.
- ☐ I remember that I am paid to solve business problems, often by writing code, not merely to write code.

3. Collaboration and teamwork

Code reviews

- ☐ I review for correctness, test coverage, maintainability, and clarity.
- ☐ I check whether a change fits the broader system.
- ☐ I look for unnecessary complexity, duplication, poor abstractions, and hard-to-maintain code.
- ☐ I keep my review tone professional, moderate, and positive.
- ☐ I ask questions before assuming the author is wrong.
- ☐ I distinguish blocking issues from nitpicks.
- ☐ I avoid excessive nitpicking and push for automation or standards where possible.
- ☐ I explain review feedback clearly, especially to new joiners.
- ☐ I consider timezone and remote-work friction when reviewing.
- ☐ I move long review threads to a direct conversation when needed.

Pairing

- ☐ I use pairing to unblock work, debug, onboard, design, and transfer knowledge.
- ☐ I clarify the problem before jumping into solutions.
- ☐ I understand the urgency before choosing how deeply to pair.
- ☐ When I know the answer, I explain rather than silently take over.
- ☐ I teach the other person how to solve similar problems next time.
- ☐ I avoid immediately giving answers when coaching would be more valuable.
- ☐ I give the other person enough space to think and type.
- ☐ I give compliments when someone works well.
- ☐ I am willing to ask for help with pairing without treating it as a weakness.

Mentoring

- ☐ I look for opportunities to mentor informally through reviews, pairing, and conversations.
- ☐ I seek mentors for my own growth.
- ☐ I clarify expectations at the start of a mentorship relationship.
- ☐ I listen to what the mentee has to say before giving advice.
- ☐ I avoid serving answers on a silver platter.
- ☐ I ask questions that help the mentee reason through the problem.
- ☐ I discuss specific situations, not only abstract advice.
- ☐ I connect mentees with people in my network when useful.
- ☐ I let mentees know I am on their side.

- ☐ I tailor mentoring for technical and non-technical topics.
- ☐ As a mentee, I bring topics, challenges, wins, reflections, and follow-ups.
- ☐ I treat mentorship as a long-term professional investment.

Giving feedback

- ☐ I give positive feedback when people do good work.
- ☐ I make praise concrete and specific.
- ☐ I avoid insincere praise.
- ☐ I ask questions before giving critical feedback.
- ☐ I deliver corrective feedback empathetically.
- ☐ I use feedback to help people grow, not to show superiority.
- ☐ I give feedback early enough to be useful.

Working with other engineering teams

- ☐ I understand which teams my team depends on.
- ☐ I know which teams depend on my team.
- ☐ I map shared ownership, APIs, infrastructure, customer-facing surfaces, and overlapping features.
- ☐ I introduce myself to key people on related teams.
- ☐ I learn what other teams own, how they work, and what challenges they face.
- ☐ I build personal connections before urgent cross-team problems arise.
- ☐ I consult owning teams before changing their code or systems.
- ☐ I use code history to identify the right person to ask.

Influence

- ☐ I ship work that balances quality and speed for my environment.
- ☐ I learn what “great work” means in my organization.
- ☐ I gather regular feedback from peers and managers.
- ☐ I get to know people outside engineering.
- ☐ I join calls or discussions with remote colleagues instead of staying fully asynchronous.
- ☐ I participate in RFCs, design docs, ADRs, or cross-team planning when I can help.
- ☐ I work on cross-team projects to build trust and understanding.
- ☐ I share progress and wins without boasting.
- ☐ I build a personal brand based on reliability, judgment, and useful impact.

4. Software engineering craft

Languages, platforms, and domains

- ☐ I am strong in at least one programming language.
- ☐ I continue learning additional languages, frameworks, and platforms.
- ☐ I understand imperative, declarative, and functional programming styles.
- ☐ I can work outside my primary platform when the project requires it.
- ☐ I understand enough about frontend, backend, mobile, infrastructure, and embedded contexts to debug cross-platform issues.
- ☐ I read code reviews and changes on other platforms to learn.
- ☐ I volunteer for small tasks outside my core stack.
- ☐ I pair with engineers who work on stacks I want to learn.
- ☐ I use AI assistants to accelerate learning, but verify their output.

Debugging

- ☐ I know my debugging tools well.
- ☐ I know where production dashboards, logs, metrics, and traces live.
- ☐ I know how to query logs and interpret production signals.
- ☐ I understand the codebase structure well enough to find relevant code.
- ☐ I map ownership and dependencies in large codebases.
- ☐ I learn global code search and useful shortcuts.
- ☐ I understand enough infrastructure to debug deployment, secrets, certificates, and configuration issues.
- ☐ I learn from outages and past incident reports.
- ☐ I offer to help investigate real outages when appropriate.

Tech debt

- ☐ I recognize tech debt as an incremental cost that compounds over time.
- ☐ I distinguish useful, temporary debt from harmful long-term debt.
- ☐ I pay down small tech debt when I encounter it.
- ☐ I quantify larger tech debt before proposing major cleanup.
- ☐ I connect tech debt work to business impact when seeking prioritization.
- ☐ I pair tech debt cleanup with high-impact projects when possible.
- ☐ I write readable code that future engineers can understand.

- ☐ I make code easy to delete.
- ☐ I design systems with extensibility in mind.
- ☐ I note small debt items instead of asking permission for every tiny cleanup.
- ☐ I avoid premature cleanup when speed is more important than polish.

Documentation

- ☐ I write design documents or RFCs before non-trivial projects.
- ☐ I document tradeoffs, diagrams, and reasoning.
- ☐ I create test, rollout, and migration plans when appropriate.
- ☐ I maintain API, interface, SDK, or integration documentation.
- ☐ I write release notes when they help other teams or users.
- ☐ I maintain onboarding documentation for systems my team owns.
- ☐ I help create or improve a team handbook.
- ☐ I write on-call runbooks for alerts and incidents.
- ☐ I update user manuals or guides when behavior changes.
- ☐ I treat documentation as high-leverage engineering work.

Scaling engineering practices

- ☐ I help the team adopt practices that improve quality and speed.
- ☐ I do not blindly copy “best practices” without considering team context.
- ☐ I use written planning for non-trivial projects.
- ☐ I support automated testing where it improves confidence.
- ☐ I consider test-driven development when useful.
- ☐ I support code reviews before production changes.
- ☐ I consider post-commit reviews for very small or highly experienced teams.
- ☐ I use testing environments where they add confidence.
- ☐ I support staged rollouts for risky changes.
- ☐ I support testing in production safely through flags, canaries, or monitoring.
- ☐ I start by asking what challenge the team is solving, not which practice is fashionable.

5. Testing

General testing mindset

- ☐ I verify that software works as expected before and after shipping.
- ☐ I use manual testing, automated testing, and production monitoring appropriately.
- ☐ I treat testing as part of engineering culture, not a separate afterthought.
- ☐ I choose tests based on system needs rather than dogma.

Unit tests

- ☐ I write fast unit tests.
- ☐ I keep unit tests reliable and deterministic.
- ☐ I keep unit tests focused and atomic.
- ☐ I use unit tests to validate behavior, document code, and prevent regressions.
- ☐ I design code with testability in mind, including manageable dependencies.
- ☐ I recognize when lack of unit tests makes refactoring risky.

Integration and UI tests

- ☐ I use integration tests to verify multiple units working together.
- ☐ I understand when integration tests are more valuable than isolated unit tests.
- ☐ I use UI or end-to-end tests to simulate real user flows.
- ☐ I recognize that UI tests are often slower and more brittle.
- ☐ I clarify what the team means by unit, integration, UI, and end-to-end tests.

Choosing a test model

- ☐ I understand the testing pyramid: many unit tests, fewer integration tests, few UI tests.
- ☐ I understand the testing trophy: strong emphasis on integration tests, especially in frontend/full-stack contexts.
- ☐ I do not force a single test model onto every system.
- ☐ I ask which tests actually catch the most valuable problems in my system.
- ☐ I ask engineers at similar companies or in similar domains what testing approaches work for them.

Specialized tests

- ☐ I consider performance tests when latency or responsiveness matters.
- ☐ I consider load tests for expected traffic spikes or capacity validation.
- ☐ I understand chaos testing as a way to validate resilience.
- ☐ I use snapshot tests where visual or UI output comparison is useful.

- ☐ I monitor app size or bundle size where it affects users.
- ☐ I maintain smoke tests for basic release confidence.
- ☐ I keep sanity/manual tests documented when automation is not practical.
- ☐ I consider accessibility, security, and compatibility testing where relevant.

Testing in production

- ☐ I use feature flags for safe production exposure.
- ☐ I use canary deployments for limited rollout.
- ☐ I understand blue-green deployments and when they help.
- ☐ I support automated rollbacks when health signals degrade.
- ☐ I use multi-tenancy or isolated production contexts when useful.
- ☐ I balance production testing with privacy, compliance, and safety constraints.
- ☐ I value production testing for confidence, debugging, and fewer test environments.

Automated testing tradeoffs

- ☐ I use automated tests to validate correctness.
- ☐ I use automated tests to catch regressions before or during deployment.
- ☐ I use tests as living documentation where appropriate.
- ☐ I use tests as a safety net for major changes.
- ☐ I account for the time required to write tests.
- ☐ I manage slow tests so they do not damage development cadence.
- ☐ I identify and fix flaky tests.
- ☐ I account for maintenance cost when changing complex tests.
- ☐ I treat automated testing as a baseline for long-lived, maintainable software.

6. Software architecture

Design documents, RFCs, and architecture docs

- ☐ I use design documents or RFCs to clarify thinking before major work.
- ☐ I use RFCs to gather feedback early.
- ☐ I choose the right RFC style: prototype first, wait for feedback, build with feedback, or document after decisions.

- ☐ I explain the context, the proposed approach, the trade-offs, and the open questions.
- ☐ I seek feedback asynchronously, synchronously, or in hybrid form depending on project complexity.
- ☐ I do not forget that the goal is to ship a good project, not to perform a perfect RFC ritual.
- ☐ I record architectural decisions in ADRs, C4 diagrams, or another lightweight format.

Prototyping and proof of concept

- ☐ I use prototypes to explore unknowns.
- ☐ I build throwaway prototypes when concrete learning is more valuable than abstract debate.
- ☐ I clearly label prototypes as non-production.
- ☐ I avoid shipping prototype code as production code without proper engineering work.
- ☐ I use prototypes to resolve uncertainty, align teams, and demonstrate feasibility.
- ☐ I choose prototyping when planning alone would be slow or speculative.

Domain-driven design

- ☐ I build a shared vocabulary with business/domain experts.
- ☐ I identify bounded contexts in complex domains.
- ☐ I model entities, value objects, aggregates, and domain events where helpful.
- ☐ I use domain modeling to reduce misunderstandings between business and engineering.
- ☐ I design code that resembles the business domain.
- ☐ I use DDD to improve readability, maintainability, and extensibility.

Shipping architecture

- ☐ I can explain the business goal behind an architecture change.
- ☐ I connect architecture work to revenue, cost, user churn, developer productivity, reliability, or another business metric.
- ☐ I seek buy-in from stakeholders before pushing major changes.
- ☐ I write proposals that explain decisions clearly and reduce ambiguity.
- ☐ I talk to key people before or during proposal writing.
- ☐ I consider whether feedback against my idea is valid business feedback.
- ☐ I break decision paralysis by specifying capability requirements.

- ☐ I identify a decision-maker before major architectural debates.
- ☐ I clarify who will own and maintain the solution.
- ☐ I use prototypes to move stuck discussions forward.
- ☐ I get product sponsorship for architecture work with business impact.
- ☐ I define what constitutes a successful rollout.
- ☐ I create and follow a rollout plan.
- ☐ I define the right time to bake or validate the change in production.
- ☐ I prepare a rollback plan.
- ☐ I run pre-mortems for risky architectural changes.
- ☐ I remember that few software decisions are truly final.
- ☐ I capture lessons from rollbacks or failed decisions.
- ☐ I reflect after shipping to learn how the architecture performs in reality.
- ☐ I share architectural learnings with others.

7. Final senior engineer takeaways

- ☐ I understand that senior engineering impact is more important than raw effort.
- ☐ I know coding is only one part of senior-level engineering.
- ☐ I collaborate efficiently with others.
- ☐ I help my team get work done.
- ☐ I mentor others and seek mentorship myself.
- ☐ I understand that “senior” means different things across companies.
- ☐ I take responsibility for unblocking complex projects.
- ☐ I ask for help when I’m stuck and bring options rather than only problems.
- ☐ I build systems, relationships, and practices that continue to pay off after my immediate task is done.